

```

1  '''prmmattress.py
2
3      This is the PRM skeleton code for the mattress moving problem.
4
5      PLEASE FINISH WRITING THE CODE, especially where marked by FIXME.
6
7  '''
8
9  import bisect
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import random
13 import time
14
15 from math          import inf, pi, sin, cos, sqrt, ceil, dist
16
17 from shapely.geometry import Point, LineString, Polygon
18 from shapely.geometry import MultiLineString, MultiPolygon
19 from shapely.prepared import prep
20
21 from vandercorput   import vandercorput
22
23 #####
24 #
25 #   Parameters
26 #
27 #   FIXME: Define the N/K...
28 #
29 N = 250 # FIXME: Select the number of nodes
30 K = 10  # FIXME: Select the number of nearest neighbors
31
32 # Define the mattress dimensions.
33 L = 7   # Legth of the mattress
34 W = 1   # Width of the mattress
35
36 # Step size in states for connectivity testing and plotting. Make
37 # sure there is no space between the physical bounding boxes in the
38 # test paths.
39 STEPSIZE = 0.2
40
41 # FIXME: Sampling Style, change in part (c)
42 SAMPLEATEDGES = True
43
44 # FIXME: Include the bonus wall for part (d)

```

```

45 BONUSWALL = False
46
47 #####
48 #
49 #   Angle Wrapping Utilities (No Fixes Needed)
50 #
51
52 # Angle Wrap Utility. Return the angle wrapped into +/- 1/2 of full range.
53 def wrap(angle, fullrange):
54     return angle - fullrange * round(angle/fullrange)
55
56 def wrap180(angle): return wrap(angle, pi)
57 def wrap360(angle): return wrap(angle, 2*pi)
58
59
60 #####
61 #
62 #   World Definitions (No Fixes Needed)
63 #
64 #   List of obstacles/objects as well as the start/goal.
65 #
66 (xmin, xmax) = (0, 30)
67 (ymin, ymax) = (0, 20)
68
69 (xA, xB, xC, xD, xE) = ( 5, 12, 15, 18, 21)
70 (yA, yB, yC, yD)      = ( 5, 10, 12, 15)
71
72 xlabel = (xmin, xA, xB, xC, xD, xE, xmax)
73 ylabel = (ymin, yA, yB, yC, yD,      ymax)
74
75 outside = LineString([[xmin, ymin], [xmax, ymin], [xmax, ymax],
76                        [xmin, ymax], [xmin, ymin]])
77 wall1    = LineString([[xmin, yB], [xC, yB]])
78 wall2    = LineString([[xD, yB], [xmax, yB]])
79 wall3    = LineString([[xB, yC], [xC, yC], [xC, ymax]])
80 bonus    = LineString([[xD, yC], [xE, yC]])
81
82 # Collect all the wall and prepare (for faster checking).
83 if BONUSWALL:
84     walls = prep(MultiLineString([outside, wall1, wall2, wall3, bonus]))
85 else:
86     walls = prep(MultiLineString([outside, wall1, wall2, wall3]))
87
88 # Define the start/goal states (x, y, theta)
89 (xstart, ystart, tstart) = (xA, yD, pi/2)
90 (xgoal, ygoal, tgoal)    = (xA, yA, 0)

```

```

91
92
93 #####
94 #
95 #   Visualization Class (No Fixes Needed)
96 #
97 #   This renders the world. In particular it provides the methods:
98 #       show(text = '')           Show the current figure
99 #       drawNode(node,           **kwargs) Draw a single node
100 #       drawEdge(node1, node2, **kwargs) Draw an edge between nodes
101 #       drawPath(path,           **kwargs) Draw a path (list of nodes)
102 #
103 class Visualization:
104     def __init__(self):
105         # Clear the current, or create a new figure.
106         plt.clf()
107
108         # Create a new axes, enable the grid, and set axis limits.
109         plt.axes()
110         plt.grid(True)
111         plt.gca().axis('on')
112         plt.gca().set_xlim(xmin, xmax)
113         plt.gca().set_ylim(ymin, ymax)
114         plt.gca().set_xticks(xlabels)
115         plt.gca().set_yticks(ylabels)
116         plt.gca().set_aspect('equal')
117
118         # Show the walls.
119         for l in walls.context.geoms:
120             plt.plot(*l.xy, 'k', linewidth=2)
121         if bonus in walls.context.geoms:
122             plt.plot(*bonus.xy, 'b:', linewidth=3)
123
124         # Show immediately.
125         self.show()
126
127     def show(self, text = ''):
128         # Show the plot.
129         plt.pause(0.001)
130         # If text is specified, print and wait for confirmation.
131         if len(text)>0:
132             input(text + ' (hit return to continue)')
133
134     def drawNode(self, node, **kwargs):
135         plt.plot(*node.outline.exterior.xy, **kwargs)
136

```

```

137     def drawEdge(self, head, tail, **kwargs):
138         plt.plot([head.x, tail.x], [head.y, tail.y], **kwargs)
139
140     def drawPath(self, path, **kwargs):
141         for i in range(len(path)-1):
142             n = ceil(path[i].distance(path[i+1]) / STEPSIZE)
143             for j in range(n):
144                 node = path[i].intermediate(path[i+1], j/n)
145                 self.drawNode(node, **kwargs)
146             self.drawNode(path[-1], **kwargs)
147
148
149     #####
150     #
151     #   A* Planning Algorithm (No Fixes Needed)
152     #
153     def astar(nodes, start, goal):
154         # Clear the A* search tree information.
155         for node in nodes:
156             node.done      = False
157             node.seen      = False
158             node.parent    = None
159             node.creach    = 0
160             node.ctogoest  = inf
161
162         # Prepare the still empty *sorted* on-deck queue.
163         onDeck = []
164
165         # Begin with the start node on-deck.
166         start.done      = False
167         start.seen      = True
168         start.parent    = None
169         start.creach    = 0
170         start.ctogoest  = start.costToGoEst(goal)
171         bisect.insort(onDeck, start)
172
173         # Continually expand/build the search tree.
174         while True:
175             # Make sure we have something pending in the on-deck queue.
176             # Otherwise we were unable to find a path!
177             if not (len(onDeck) > 0):
178                 return None
179
180             # Grab the next node (first on deck).
181             node = onDeck.pop(0)
182

```

```

183         # Mark this node as done and check if the goal is thereby done.
184         node.done = True
185         if goal.done:
186             break
187
188         # Add the neighbors to the on-deck queue (or update)
189         for neighbor in node.neighbors:
190             # Skip if already done.
191             if neighbor.done:
192                 continue
193
194             # Compute the cost to reach the neighbor via this new path.
195             creach = node.creach + node.costToConnect(neighbor)
196
197             # Just add to on-deck if not yet seen (in correct order).
198             if not neighbor.seen:
199                 neighbor.seen = True
200                 neighbor.parent = node
201                 neighbor.creach = creach
202                 neighbor.ctogoest = neighbor.costToGoEst(goal)
203                 bisect.insort(onDeck, neighbor)
204                 continue
205
206             # Skip if the previous path to reach (cost) was same or better!
207             if neighbor.creach <= creach:
208                 continue
209
210             # Update the neighbor's connection and resort the on-deck queue.
211             # Note the cost-to-go estimate does not change.
212             neighbor.parent = node
213             neighbor.creach = creach
214             onDeck.remove(neighbor)
215             bisect.insort(onDeck, neighbor)
216
217         # Build the path.
218         path = [goal]
219         while path[0].parent is not None:
220             path.insert(0, path[0].parent)
221
222         # Return the path.
223         return path
224
225
226 #####
227 #
228 #   Node Definition (to be fixed - see FIXME)

```

```

229 #
230 class Node():
231     def __init__(self, x, y, t):
232         # Define/remember the state (x,y,theta).
233         self.x = x
234         self.y = y
235         self.t = t                # Angle theta
236
237         # Precompute the mattress box (for collision detection and drawing).
238         self.outline = Polygon(
239             [[x + L/2*cos(t) + W/2*sin(t), y + L/2*sin(t) - W/2*cos(t)],
240              [x + L/2*cos(t) - W/2*sin(t), y + L/2*sin(t) + W/2*cos(t)],
241              [x - L/2*cos(t) - W/2*sin(t), y - L/2*sin(t) + W/2*cos(t)],
242              [x - L/2*cos(t) + W/2*sin(t), y - L/2*sin(t) - W/2*cos(t)]]
243
244         # Edges = set of neighbors. This needs to be filled in.
245         self.neighbors = set()
246
247         # Clear the status, connection, and costs for the A* search tree.
248         # TRUNK: done = True
249         # LEAF: done = False, seen = True
250         # AIR: done = False, seen = False
251         self.done = False
252         self.seen = False
253         self.parent = None
254         self.creack = 0                # Known/actual cost to get here
255         self.ctogoest = inf            # Estimated cost to go from here
256
257     #####
258     # A* functions:
259     # Actual cost to connect to a neighbor and estimated to-go cost to
260     # a distant (goal) node.
261     def costToConnect(self, other):
262         return self.distance(other)
263
264     def costToGoEst(self, other):
265         return self.distance(other)
266
267     # Define the "less-than" to enable sorting in A*. Use total cost estimate.
268     def __lt__(self, other):
269         return (self.creack + self.ctogoest) < (other.creack + other.ctogoest)
270
271     #####
272     # PRM functions:
273     # Compute the relative distance to another node.
274     def distance(self, other):

```

```

275         # FIXME_FIRST: Compute and return the distance (used to sort).
276
277         # Set the radius of mattress which is L / 2
278         R = L / 2
279
280         # Find change in angle when 0 = 180 = 360 degrees
281         angle_distance = abs(sin(wrap180(self.t - other.t)))
282
283         # Use formula to find distance between two nodes accounting for change in x, y, and
284         distance = sqrt((self.x - other.x)**2 + (self.y - other.y)**2 + (R * angle_distance))
285
286         return distance
287
288         # Check whether in free space.
289         def inFreespace(self):
290             return walls.disjoint(self.outline)
291
292         # Check the local planner - whether this connects to another node.
293         def connectsTo(self, other):
294             n = ceil(self.distance(other) / STEPSIZE)
295             for delta in vandercorput(1/n):
296                 if not self.intermediate(other, delta).inFreespace():
297                     return False
298             return True
299
300         #####
301         # Utilities:
302         # In case we want to print the node.
303         def __repr__(self):
304             return ("" % (self.x, self.y, self.t))
305
306         # Compute/create an intermediate node. This can be useful if you
307         # need to check the local planner by testing intermediate nodes.
308         def intermediate(self, other, alpha):
309             # Return the current Node
310
311             new_x = self.x + alpha * (other.x - self.x)
312             new_y = self.y + alpha * (other.y - self.y)
313             new_t = wrap180(self.t + alpha * (wrap180(other.t - self.t)))
314
315             return Node(new_x, new_y, new_t)
316
317         #####
318         #
319         #   PRM Functions (to be fixed - see FIXME)
320         #

```

```

321 # Create the list of nodes.
322 def createNodesUniform(N):
323     # FIXME: create, see also last week's code
324
325     # Add nodes sampled uniformly across the space.
326     nodes = []
327     while len(nodes) < N:
328         node = Node(random.uniform(xmin, xmax),
329                     random.uniform(ymin, ymax),
330                     random.uniform(-pi/2, pi/2))
331         if node.inFreespace():
332             nodes.append(node)
333     return nodes
334
335
336 def createNodesAtEdges(N):
337
338     # FIXME: please add in part (c)
339
340     r = W / 2
341     R = L / 2
342     nodes = []
343
344     while len(nodes) < N:
345         # Reset r before generating new node
346         r = W / 2
347
348         # Generate a random Node
349         node = Node(random.uniform(xmin, xmax),
350                     random.uniform(ymin, ymax),
351                     random.uniform(-pi/2, pi/2))
352
353         # Continue generating new node until it collides with obstacle
354         while node.inFreespace():
355             node = Node(random.uniform(xmin, xmax),
356                         random.uniform(ymin, ymax),
357                         random.uniform(-pi/2, pi/2))
358
359         # Continue loop until a node by edge is found
360         while True:
361             dx = random.uniform(-r, r)
362             dy = random.uniform(-r, r)
363             dt = random.uniform(-r/R, r/R)
364             test_node = Node(node.x + dx, node.y + dy, wrap180(node.t + dt))
365
366             # If non-colliding node (node by edge) is found, add to list

```



```

367         # else, increase radius by 10%
368         if test_node.inFreespace():
369             nodes.append(test_node)
370             break
371         else:
372             r = 1.1 * r
373
374     return nodes
375
376
377     # Connect to K neighbors (from all, testing nearest first)
378 def connectKNeighbors(nodes, K):
379     # FIXME: copy from the previous planner code
380
381     # Clear any existing neighbors, which are stored as a set.
382     for node in nodes:
383         node.neighbors = set()
384
385     # Check the neighbors.
386     for node in nodes:
387         # Examine the K nearest neighbors (ignoring node itself).
388         indicies = np.argsort(np.array([node.distance(n) for n in nodes]))
389
390         # Define total connections for each node and current neighbor
391         total_connect = 0
392         neighbor_index = 1
393
394         # Check if total connections has reached K neighbors for given node
395         while total_connect < K and neighbor_index < len(nodes):
396             neighbor = nodes[indicies[neighbor_index]]
397             # Force a undirected graph, so node is also neighbor's neighbor.
398             if (neighbor not in node.neighbors) and (node.connectsTo(neighbor)):
399                 node.neighbors.add(neighbor)
400                 neighbor.neighbors.add(node)
401                 total_connect += 1          # If connection successful,
402                                           # increase number of connections by 1
403             neighbor_index += 1          # Move to next neighbor in list
404
405     # Compute the path cost
406 def pathCost(path):
407     # FIXME: copy from the previous planner code
408
409     cost = 0
410     for i in range(1, len(path)):
411         cost += path[i-1].costToConnect(path[i])
412     return cost

```

```

413
414 # Post process the path
415 def postProcess(path):
416     # FIXME: copy from the previous planner code
417
418     i = 1
419     while i < (len(path) - 1):          # Don't include the start and goal nodes
420         predecessor = path[i - 1]
421         successor = path[i + 1]
422         if (predecessor.connectsTo(successor)):
423             path.pop(i)                # Pop the node, but don't increment i
424                                         # since next node will fall into its place
425         else:
426             i += 1
427
428     finalpath = path
429     return finalpath
430
431
432 #####
433 #
434 # Run/Test/Main Code (No Fixes Needed)
435 #
436 def run():
437     # Report the parameters.
438     print('Running with', N, 'nodes and', K, 'neighbors.')
439
440     # Create the figure. Some computers seem to need an additional show()?
441     visual = Visualization()
442     visual.show()
443
444     # Create the start/goal nodes.
445     startnode = Node(xstart, ystart, tstart)
446     goalnode = Node(xgoal, ygoal, tgoal)
447
448     # Show the start/goal nodes.
449     visual.drawNode(startnode, color='orange', linewidth=3)
450     visual.drawNode(goalnode, color='purple', linewidth=3)
451     visual.show("Showing basic world")
452
453
454     # Create the list of nodes.
455     print("Sampling the nodes...")
456     tic = time.time()
457     if SAMPLEATEDGES: nodes = createNodesAtEdges(N)
458     else:              nodes = createNodesUniform(N)

```

```

459     toc = time.time()
460     print("Sampled the nodes in %fsec." % (toc-tic))
461
462     # Show the sample nodes.
463     # if True:
464     #     for node in nodes:
465     #         visual.drawNode(node, color='grey', linewidth=1)
466     #     visual.show("Showing the nodes")
467
468     # Add the start/goal nodes.
469     nodes.append(startnode)
470     nodes.append(goalnode)
471
472
473     # Connect to the nearest neighbors.
474     print("Connecting the nodes...")
475     tic = time.time()
476     connectKNeighbors(nodes, K)
477     toc = time.time()
478     print("Connected the nodes in %fsec." % (toc-tic))
479
480     # Show the neighbor connections.
481     # if True:
482     #     for (i,node) in enumerate(nodes):
483     #         for neighbor in node.neighbors:
484     #             if neighbor not in nodes[:i]:
485     #                 visual.drawEdge(node, neighbor, color='g', linewidth=1)
486     #     visual.show("Showing the full graph")
487
488
489     # Run the A* planner.
490     print("Running A*...")
491     tic = time.time()
492     path = astar(nodes, startnode, goalnode)
493     toc = time.time()
494     print("Ran A* in %fsec." % (toc-tic))
495
496     # If unable to connect, show the part explored.
497     if not path:
498         print("UNABLE TO FIND A PATH")
499         for node in nodes:
500             if node.done:
501                 visual.drawNode(node, color='r', linewidth=1)
502         visual.show("Showing DONE nodes")
503         return
504

```

```

505     # Show the path.
506     cost = pathCost(path)
507     visual.drawPath(path, color='r', linewidth=1)
508     visual.show("Showing the raw path (cost/length %.1f)" % cost)
509
510
511     # Post process the path.
512     finalpath = postProcess(path)
513
514     # Show the post-processed path.
515     cost = pathCost(finalpath)
516     visual.drawPath(finalpath, color='b', linewidth=1)
517     visual.show("Showing the post-processed path (cost/length %.1f)" % cost)
518
519
520 def test():
521     # Define the test paths.
522     path1 = [Node( 5, 2, 0), Node( 5, 8,      pi)]
523     path2 = [Node(15, 5, 0), Node(15, 5, 1.49*pi)]
524     path3 = [Node(26, 2, 0), Node(23, 5, 0.5 *pi)]
525
526     # Create the figure. Some computers seem to need an additional show()?
527     visual = Visualization()
528     visual.show()
529
530     # Show path with end points highlighted.
531     def show(path, **kwargs):
532         visual.drawNode(path[0 ], linewidth=2, **kwargs)
533         visual.drawNode(path[-1], linewidth=2, **kwargs)
534         visual.drawPath(path,      linewidth=1, **kwargs)
535
536     show(path1, color='b')
537     show(path2, color='g')
538     show(path3, color='r')
539     visual.show("Showing the test paths")
540
541
542 if __name__ == "__main__":
543     test()
544     run()

```